

Assignment - 07

Q1.

→ A local variable is created inside a function & exists only during that function's execution. It is stored in function's local namespace & is inaccessible once the function returns.

A global variable is defined outside all functions, it belongs to the module's global namespace and can be read & accessed by any function.

eg- `x = 100` `# Global.`

```
def demo():  
    y = 50  
    print(x)              # 100  
    print(y)              # 50
```

```
demo()  
print(x)              # 100  
print(y)              → ERROR
```

Q2.

- Disadvantages of global variables -
- 1) Hidden side effects.
 - 2) Difficult debugging & testing. testability.
 - 3) Makes data flow implicit.
 - 4) Reduced modularity.
 - 5) Anyone can access & alter it.

Q3.

→ The `global` keyword tells Python -

"Do not create a local variable, Modify the existing global variable."

It is rarely used, as global variables are avoided without global, python throws UnboundLocalError.

Q4.

→ `--name--` is a special built-in module variable which is automatically created by PVM. It stores name of current module / file.
We can't assign it explicitly.

Q5.

→ `--main--` indicates that the current .py file is the entry point of program execution.
So PVM, assigns every file which isn't a module `--name--` as `--main--`.

Q6.

→ In python, funcs are first-class objects.

That means a function can -

i) be assigned to variables.

ii) passed as arguments

iii) stored in d.s.

iv) returned to another func().

Use-case:

i) Decorators.

ii) Callback functions.

iii) Closures.

Q7.

→ A module is a .py file containing set of functions, classes etc.
These are used by other files using import keyword.

Module → contains lots of functions. / entire file.
functions → Reusable block of code.

Q8.

→ When a .py is executed directly,
for eg - we run this command - `python demo.py`
↳ then PVM assigns `--name-- = "__main__"`

When same file is imported into another file,
lets say - `first.py` → `import demo`

↳ then PVM assigns name as `demo` to `demo.py` &
now `first.py` → becomes → `--main--`

Q9.

→ `if ("__main__" == __name__):` → This ensures that code runs only when the file is executed directly. → ① Same file acts as imported file, an exe. program.

Q10.

→ ~~creat~~ `return x` → object `x` is returned internally
`return x, y` → Tuple `(x, y)`
`return` → None